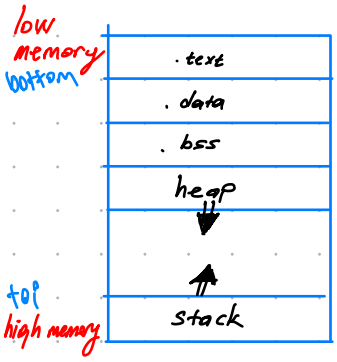
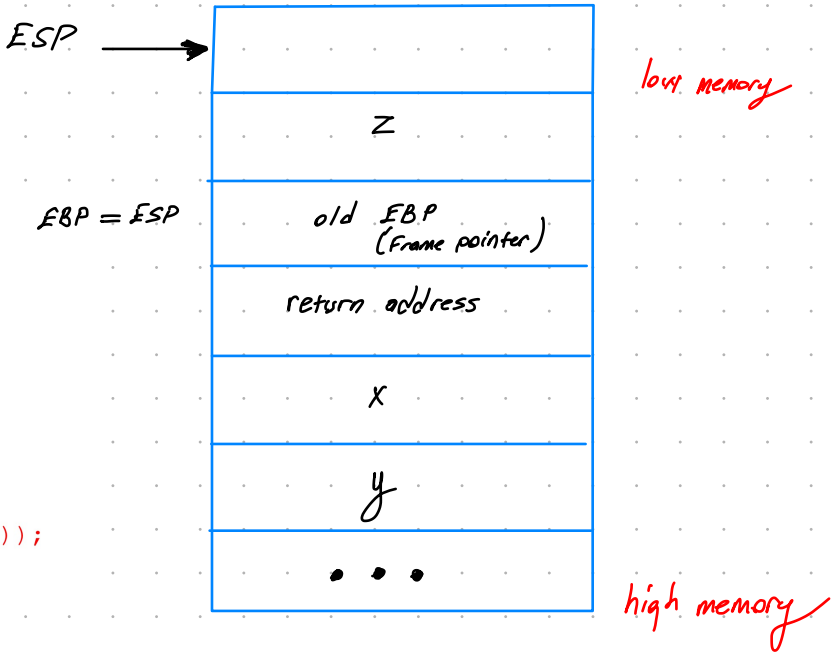
Compilation and Memory Layout



Stack frames; environment for a function call

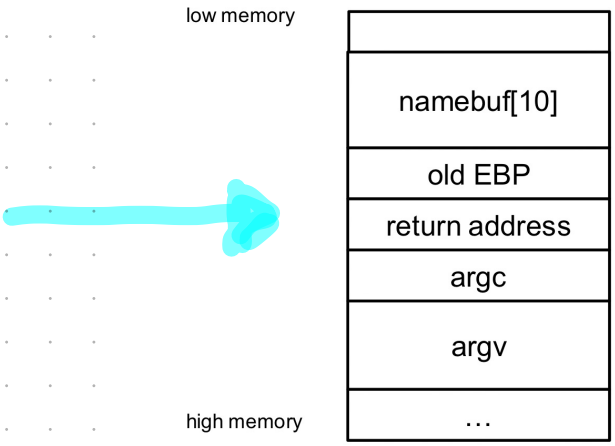
* Stack grows "downwards" i.e towards low memory
* Holds local variables, temporary values, and return addresses.

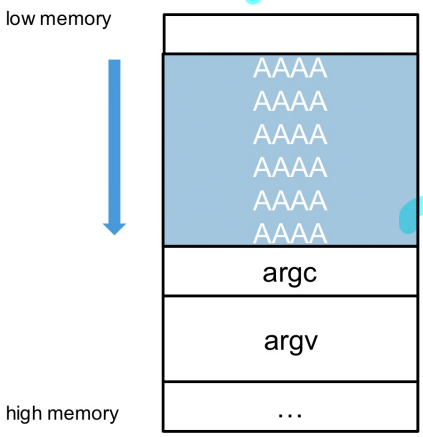



```c
#include <stdio.h>
#include <stdlib.h>

int sum(int x, int y){
      int z = x+y;
      return z;
}
int main(int argc, char* argv[]){
      int x = atoi(argv[1]);
      int y = atoi(argv[2]);
      printf("x + y = %d\n",sum(x,y));
      return 0;
}
```

Buffer Overflow

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]){
      char namebuf[10];
      strcpy(namebuf, argv[1]);
      printf("Hello, %s!\n", namebuf);
      return 0;
}
```





Overrunning/Overflowing the buffer boundaries means to write into adjacent memory locations.

Segmentation Fault: accessing a memory that we supposed to access.

Why it works; because stack grows top to bottom

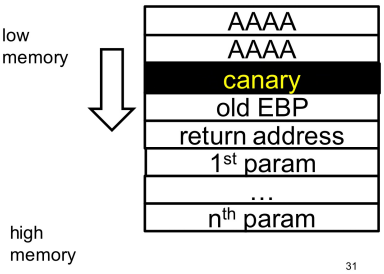
Summary: Why Buffer Flow Attacks work;

1.) Stack grows top to bottom which means overwriting is possible

2-) We do not distinguish code and data; stack should not contain any code!

Defenses against Buffer Overflow

* Canary:
  * By inserting a small value known as a canary between the stack variables (buffers) and the function return address. When a stack-buffer overflows into the function return address, the canary is overwritten. During function return the canary value is checked and if the has changed the program is terminated.

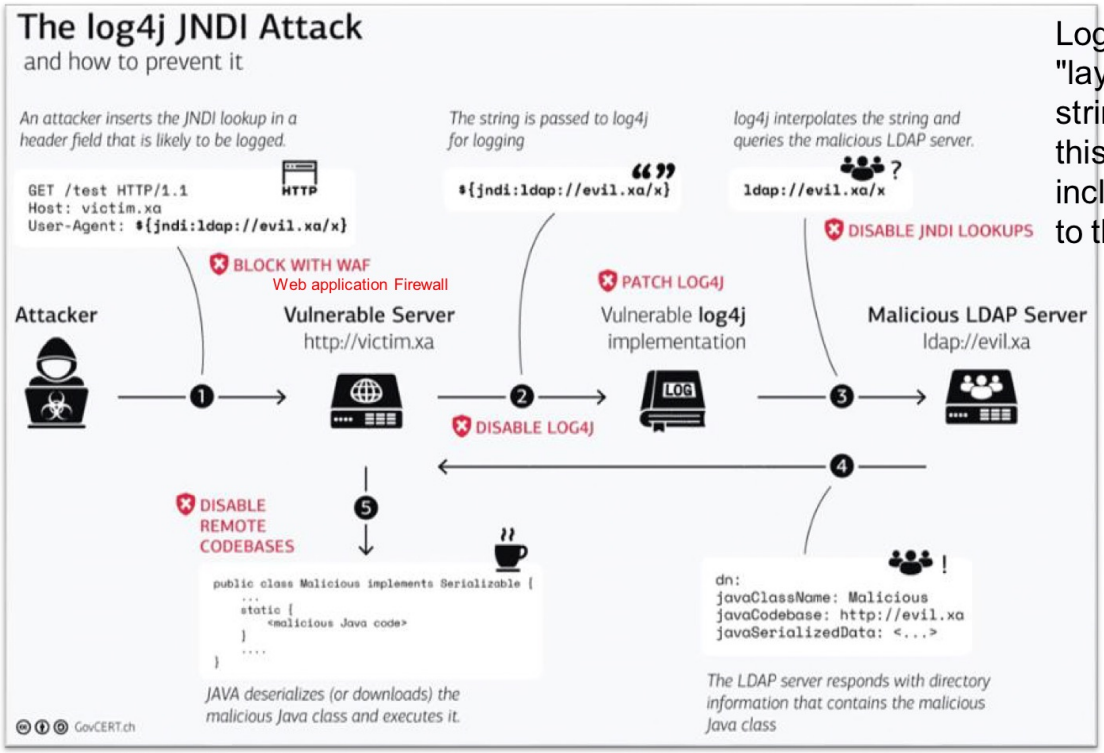


* Avoid C/C++ when possible
* Defensive Programming; use libraries such as Safe C.
* System Based Defenses
  ↳ Address Space Layout Randomization (ASLR)

* Pegasus (NSO zero-click iMessage Exploit)

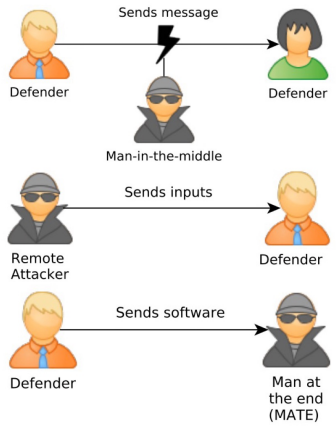
* Log4j

og4Shell



Logging includes "layouting" the string to log; this reaction may include a call to the LDAP server

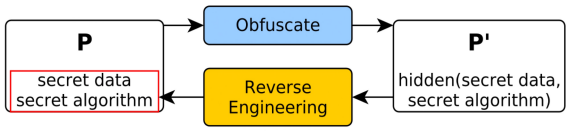
Obfuscation



1-) Man-in-the-middle
* Attack comm. channels

2-) Remote Attacker
* exploit vulnerabilities

3-) Man-at-the-end (MATE)
* reverse engineering the software



Categorization of Obfuscation

* Static vs Dynamic

* Point of Insertion ; source code, intermediate rep., assembly code

* Transformation Targets ;
  - ↳ layout: scramble identifiers and code layout
  - ↳ data : obfuscate data (structures) embedded in code
  - ↳ Control Flow: obfuscate secret algorithms

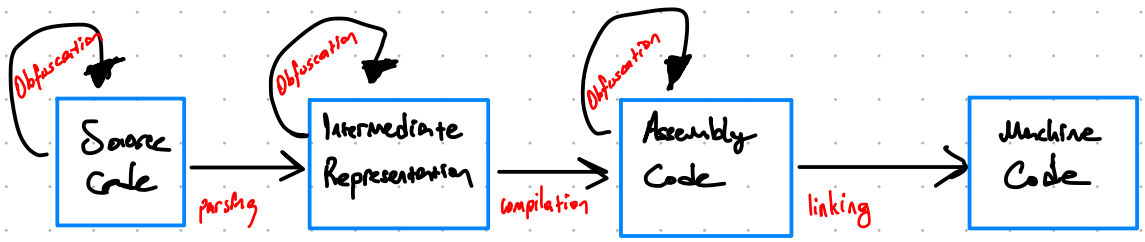
Static Obfuscation Transformations

* Scramble identifiers
* Instruction substitution
* Garbage code insertion
* Merging and splitting functions
* Opaque predicates
* Control-flow flattening
* Virtualization Obfuscation

Protect Intellectual Property

* Opaque expressions
* White-box cryptography

Protect secret data



Virtualization Obfuscation

Goal: Hide secret algorithm of program P.

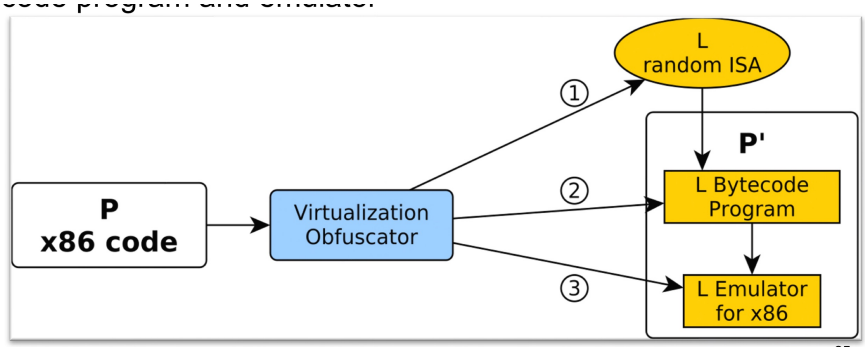
Obfuscation Procedure

1.) Generate random bytecode instruction set architecture (ISA) L covering all instructions of P.

2.) Translate P to L bytecode program.

3.) Generate emulator to interpret L bytecode on x86 machine.

Output: Obfuscated program (P') consists of bytecode program and emulator.



25

Strengths

* Random ISA, different for each instance
* Control-flow flattening → Different locations in original program mapped to same case branch
* Many other obfuscation techniques can be added on top of
  * Each case branch
  * The switch statement
  * The data and code arrays

White-Box Cryptography (WBC)

* **Goal:** Hide encryption/decryption key
* **Idea:**
  * Embed the key inside the cipher (e.g in S-boxes)
  * Convert cipher communication into large network of look-up tables (LUTs)

Attack:

WB-AES [6] and WEB-DES broken